

Succeeding as a Zoox Compute Test Engineer

The First Year — Judgment, Relationships, and Impact

May 2026

Contents

1	How to Use This Guide	2
2	The First 90 Days (Relationships and Ownership)	3
2.1	Days 1–30: Earn trust, learn the system, change nothing big	3
2.2	Days 31–60: Own a corner, contribute visibly	3
2.3	Days 61–90: Demonstrate independent judgment	3
3	The Org Map: Who You Work With and How	4
4	Ramping Without Breaking the Line	5
5	The Senior Test Engineer Mindset	6
6	A Systematic Debugging Method	7
7	Working With Contract Manufacturers	8
8	Communication and Influence	9
9	Continuous Improvement as a Discipline	10
10	What the JD Implies But Doesn't Say	11
10.1	Lab instrumentation & physical-layer measurement (<i>hidden in “control test instruments”</i>)	11
10.2	Software engineering at scale (<i>hidden in “develop scripts and code”, “build and release”</i>)	11
10.3	Data systems, statistics & continuous improvement (<i>hidden in “analyze results”, “yield”</i>)	11
10.4	Working across all four phases & influencing the design (<i>hidden in “all phases”, “validate new products”</i>)	11
10.5	Reliability/stress integration (<i>implied by “all phases” + safety-critical compute</i>)	12
10.6	Safety-criticality mindset & traceability (<i>inherent to a robotaxi role; not in the JD</i>)	12
10.7	Behavioral / seniority expectations (<i>hidden in “work independently, manage priorities”</i>)	12
11	Landing the Tools You Bring	13
12	Pitfalls to Avoid	14
13	Living the Leadership Principles (For a Test Engineer)	15
14	A Learning Plan for the Platform	16
15	What “Good” Looks Like	17

1 How to Use This Guide

Guide A makes you technically dangerous on day one. This guide is about everything *around* the technical work that actually determines whether you succeed: how you ramp, who you work with, how you debug under pressure, how you support contract manufacturers, how you communicate findings, how you drive improvement, and how you land the tools you bring without overstepping.

You are a senior hire (8+ years; you have ~13). That changes the job in a specific way: nobody is going to hand you tickets. Success is measured by judgment and ownership — can you take an ambiguous problem (“this new board needs test coverage”) and turn it into a deployed, trusted, fast test program with the right people bought in. The technical skill is table stakes; the senior skill is everything in this guide.

The one sentence for this guide: *Earn trust fast by shipping small correct things, then use that trust to make the test process measurably better — without breaking the line or the relationships while you do it.*

A grounding note for your situation. You’ve done this transition before, just at smaller scale: you walked into 2G with no controls background and built production servo systems; you built a station dashboard nobody asked for and got it adopted. Those instincts — self-direction, build-to-learn, solve-the-real-problem — are exactly right here. The new variables are scale (volume manufacturing, multiple sites), stakes (a robotaxi’s compute is safety-critical), and interfaces (more teams, contract manufacturers). This guide is about scaling what you already do well.

2 The First 90 Days (Relationships and Ownership)

Guide A has the *technical* ramp. This is the *people-and-judgment* ramp. They run in parallel.

2.1 Days 1–30: Earn trust, learn the system, change nothing big

Your only goals this month are to become useful at the existing process and to map the human and technical system. Resist the urge to redesign anything yet — you don’t have the context, and “the new senior person who immediately wanted to rewrite everything” is a reputation that takes a year to undo.

- Learn the existing test program cold before proposing changes to it. Run it, read it, trace a unit through it, find its failure history in the data. Assume it’s the way it is for reasons you don’t see yet — then ask *why*, genuinely curious, not leading.
- Meet your interfaces (the org map below): your manager, the senior test engineers, the EE and SW/FW leads for compute, the NPI/manufacturing folks, quality. For each: what do they need from test, what frustrates them about test today, how do they like to communicate.
- Find the data and the dashboard. Where do results live, what does yield look like, what are the top failure modes right now. You think in data; start swimming in theirs.
- Ship one tiny thing. A clearer error message, a fixed flaky test, a small log improvement — landed through the real release process. The point isn’t the impact; it’s proving you can move something through the system correctly and safely.

2.2 Days 31–60: Own a corner, contribute visibly

- Take ownership of one interface or test area (PCIe is the natural fit). Become the person people ask about it.
- Make a real, data-backed improvement — a coverage gap you can prove matters, or a runtime win. Propose it with numbers, build it, correlate it on golden units, release it.
- Start being the debugger. When a hard failure shows up in your area, take it, work it to root cause, and write up what you found clearly. Each closed bug is trust deposited.

2.3 Days 61–90: Demonstrate independent judgment

- Run a continuous-improvement project end to end — pick it from the data (top Pareto failure, slow step, missing coverage), drive it, measure the before/after, present the win.
- Support a prototype/new-HW bring-up with EE. This is the senior signal: you can stand up coverage for hardware that has no test yet.
- Have a point of view on where compute test should go (more margining-based limits, better CM correlation, faster soaks via confidence targets) — informed by 90 days of context, not day-one opinions.

What success looks like at 90 days: your manager trusts you with an ambiguous, important problem and doesn’t worry about it. That’s the whole game.

3 The Org Map: Who You Work With and How

You can't be effective without knowing who owns what and what each group needs from you. Approximate map (titles vary):

Group	What they own	What they need from you	How to work with them
Your test team (mgr, senior test engineers)	The test programs, stations, framework	A teammate who ships and doesn't break the line	Learn their conventions; ship small first; ask before refactoring shared code
Electrical Engineering (EE) / HW design	Schematics, layout, the boards	Sharp failure evidence; bring-up support	Speak their language (rails, lanes, SI); hand them decoded data, not "it failed"
Software / Firmware / BSP	Linux image, drivers, firmware	Tests that work on their image; clear repro of driver/FW bugs	Pin versions; when test breaks after an image update, isolate test-vs-image cleanly
NPI / Manufacturing / Operations	Running tests on the line	Fast, robust, operator-proof tests; quick failure triage	They're your users; design for them; respond fast when the line is down
Quality / Reliability	Yield, RMAs, corrective action	Data, traceability, root-cause rigor	Give them parameters not just verdicts; help close the loop on field returns
Contract Manufacturers (CMs)	Building boards at volume	Released, documented, correlated test programs; remote support	Robustness + logs + correlation; treat them as partners (whole section below)
Program / Project Management	Schedule, deliverables	Honest status, early risk flags	No surprises; flag schedule risk the moment you see it, with options

The relationship that most defines a compute test engineer is with EE. When your test finds a failure, EE decides whether it's a real defect, a test problem, or a design issue. The quality of your evidence sets the speed and tone of that conversation. "PCIe failed" puts the burden on you to prove it. "Lane 7 shows 40 Replay-Timer correctable errors/min that appear only above 70 °C, here's the margining sweep and the rail scope" puts a defect in front of EE that's hard to wave off. Be the test engineer whose failures EE believes.

4 Ramping Without Breaking the Line

A robotaxi compute line is a live, safety-relevant production system. The fastest way to lose trust is to break it — a bad test release that false-fails good units halts a line and costs real money and credibility. The discipline:

- Read-only first. Everything you do in week one is observation. Read code, read data, watch tests run. Don't touch a register, a limit, or a config on a production station until you understand the blast radius.
- Sandbox before production. Have a dev station / spare unit. Reproduce, experiment, and validate there. Never debug a hypothesis on a production line.
- Respect the release process. There's a reason test programs are versioned and gated. Use it even when your change is "obviously fine." The one time you skip it is the time it isn't.
- Golden-unit gate. Before any release, confirm your change still passes known-good units and fails known-bad ones. A test that's stricter than you intended (false fails) or looser (escapes) shows up here, not on the line.
- Change one thing, measure, then the next. When debugging or tuning, single-variable discipline. The temptation under line-down pressure is to change five things; that's how you "fix" it without knowing why and have it return next week.
- Know how to roll back. Every release should be revertible. Knowing you can undo lets you move faster safely.

Your "restart-from-any-keyword" snapshot feature at 2G is the same instinct applied to operators: design for recovery, assume things will fail mid-process, make the safe path the easy path. Bring that instinct to how you release and operate, not just how you code.

5 The Senior Test Engineer Mindset

A few principles that separate senior test judgment from script-writing. These are the things to *say* and *live*, because they're what the manager, senior engineer, and director interviews were really probing for — and what the job rewards.

Ship only good units. The test exists to protect the vehicle and the brand from a marginal board. When you're tempted to loosen a limit to recover yield, the question is “does this let a real defect through?” Yield matters, but never by hiding escapes — those come back 10,000× more expensive, and in a robotaxi they're a safety issue.

Coverage, runtime, yield is a triad you balance, not maximize. More coverage costs runtime and can cost yield (more chances to fail); faster runtime can cost coverage; looser limits buy yield but risk escapes. Every test decision moves these three. Seniority is making that trade *consciously and with data*, and being able to explain the trade you chose.

Data over opinion, always. “I think the limit should be 50 mV” loses to “the fleet distribution is 20 ± 5 mV, Cpk against a 50 mV limit is 2.0, here's the histogram.” Capture parameters, build the distribution, set the limit from it. This is your superpower — you already think this way (the bilinear torque model, R^2 thresholds). Apply it to limits.

A failure is a question, not a verdict. When a unit fails, the job isn't “mark it bad” — it's “what does this tell us.” Is it a real defect (which kind, where), a marginal unit, a test problem, or a process drift? Good test engineers treat every fail as information about the *process*, not just the unit.

The cheapest defect is the one caught earliest. Internalize the 10× curve (Guide A). Your instinct should always be “what's the earliest phase that can catch this,” and a low-grade discomfort whenever a defect is found later than it could have been.

Test the thing, not the test. Beware tests that pass because they don't actually exercise the failure mode (a “BERT” on an idle link, a thermal test that never gets hot, a link test that checks enumeration but not error counters under load). A test that can't fail isn't a test. Always ask: what defect would this catch, and have I proven it catches it?

6 A Systematic Debugging Method

Guide A gave you the *reflex*; here's the explicit *method* for when a board fails and people are waiting. It works because it's evidence-driven and avoids the two failure modes of debugging under pressure: guessing, and changing many things at once.

1. Reproduce and define “failed.” Get a precise, repeatable failure. “Sometimes flaky” is not yet a bug — pin down the exact symptom, rate, and conditions (temperature? load? specific unit? specific lane/port?). If you can't reproduce it, your first job is making it reproducible.
2. Read what the system already recorded. `dmesg`, the test logs, the counters (AER, ECC, EDAC, SMART). Most hardware failures already told you what happened; read before you poke.
3. Localize by layer and by swap. Use the layer the error names (Guide A's AER/XID/error tables) to pick physical vs protocol. Then isolate by swapping: known-good unit in the failing slot, failing unit in a known-good slot — does the failure follow the unit or stay with the station/slot? This one technique resolves a huge fraction of “is it the board or the fixture” questions.
4. Form one hypothesis and test it with one change. “It's thermal” → soak hot and watch the counter. “It's the rail” → scope it under load. Single variable. Predict what you'll see *before* you look; a surprise is more informative than a confirmation.
5. Measure at the physical layer when counters run out. Scope the rail/ripple/sequencing, margin the lane, check termination. The counters tell you *what*; the instruments tell you *why*.
6. Get to a root cause you can name and a fix you can verify. Not “reseated it and it went away” — *why* did reseating fix it (connector contact? specific lane?), and does the fix hold across thermal cycles and multiple units. “Fixed, cause unknown” means it'll be back.
7. Write it down so the next person doesn't re-walk it. A short, clear writeup with the evidence. This is how a debug becomes institutional knowledge and how you build a reputation as the person who actually closes things.

The trap to avoid under line-down pressure is shotgunning — changing several things to make it go away. It sometimes “works” and always leaves you without a root cause, so it comes back. Slower-but-single-variable beats fast-but-guessing almost every time.

7 Working With Contract Manufacturers

The JD calls this out explicitly — “*build and release test solutions for use on the manufacturing lines at Zoox and at contract manufacturing partners.*” For a senior test engineer this is one of the highest-leverage and least-glamorous parts of the job, and it’s where robustness and communication matter more than cleverness.

Why CMs exist: Zoox doesn’t build every board in-house at volume. Contract manufacturers (the EMS companies — think Flex, Jabil, and the like) have the lines, capacity, and cost structure to build at scale. You release your test programs *to them*, and they run your tests on units you may never physically touch.

What changes when the line is 2,000 miles away:

- Robustness is everything. A test that occasionally hangs is an annoyance at your bench and a line-stopping incident at a CM where you can’t walk over and fix it. Defensive coding, timeouts on every instrument/DUT call, clear failure states, and graceful recovery aren’t nice-to-haves — they’re the difference between a 10-minute issue and an overnight outage across a time zone.
- Logs are your eyes. You debug CM failures through logs and remote access. Every test must log enough — measured values, decoded errors, dmesg snippets, versions — that you can diagnose a failure from the record alone. The “attach the evidence to the failure” habit from Guide A is non-negotiable here.
- Documentation and training. The CM’s operators and engineers run your program. They need setup docs, a clear runbook, fixture instructions, and a triage guide for common fails. You’ll train remotely; assume nothing is obvious.
- Correlation and golden units. The same unit must produce the same result at Zoox and at the CM — that’s cross-site gauge reproducibility. You ship golden units (characterized known-good and known-bad) to the CM and correlate: do their stations agree with yours? A correlation gap means a fixture, calibration, or environment difference you must resolve before trusting their yield.
- Their yield is partly your responsibility. A test that false-fails at the CM costs them throughput and costs you credibility; an escape that ships from the CM is your coverage gap. You own the test even where you don’t own the line.

The release package you hand a CM is the artifact that defines this relationship: versioned test code + config, setup/runbook docs, fixture spec, acceptance criteria, golden-unit correlation data, and a failure-triage guide. Treat assembling it as a core deliverable, not an afterthought.

The on-call / line-down reflex. A CM line stop is a cross-time-zone incident — the line is down, money is burning, and you can’t walk over. The same things that make a test robust make the incident short: timeouts on every instrument/DUT call so nothing hangs forever, clear failure states so the operator’s screenshot tells you the layer, logs rich enough to diagnose from the record, and a triage guide that lets *their* engineer resolve the common 80% without waking you. The difference between a 10-minute issue and an overnight outage is almost entirely robustness + logs + documentation you built in advance, not heroics at 3 a.m. Build for the incident you won’t be awake for.

Practical reality: time zones, language differences, and you-can’t-see-it. Over-communicate in writing, make specs unambiguous, and build tests that explain themselves. The senior move is making the CM *successful and self-sufficient*, not dependent on a call to you for every failure.

8 Communication and Influence

Most of your impact flows through other people — EE fixing a design, SW fixing a driver, ops adopting a test, a CM running it right. How you communicate determines whether that happens.

Lead with evidence, structured for the reader. A failure report to EE: symptom → conditions → evidence (decoded counters, margining, scope) → what it points to → what you need from them. A yield update to your manager: the number, the trend, the top Pareto cause, what you're doing about it. Tailor the altitude — EE wants the lane and the rail; your director wants the yield and the risk.

Make the data visible. Your FastAPI station dashboard at 2G is the template: you turned “operators running between 4 stations” into “one engineer watching a browser,” and idle-after- failure time dropped from 30+ min to under 5. That's influence through tooling — you didn't argue for better monitoring, you *built* it and let it sell itself. Look for the Zoox-scale version of that (fleet yield/runtime visibility, failure-mode dashboards) once you've earned the context to build it.

Influence without authority. As a senior individual contributor you'll often need EE or SW to do something and have no authority to make them. What works: bring data not opinions, frame it as *their* problem solved (a defect they'd want caught, a driver bug repro'd cleanly), and make the ask small and specific. The dashboard story is the pattern — solve the real problem, show it, let adoption follow.

Disagree and commit. You'll sometimes lose an argument about a limit or a coverage call. State your case once, clearly, with data; if the decision goes the other way, commit to it fully and revisit with new data later if you're proven right. Re-litigating lost decisions burns the trust you need for the next one.

Write it down. Findings, root causes, decisions, runbooks. Written artifacts scale across sites and time zones and outlive the conversation. The test engineer whose investigations are documented becomes the source of truth; the one who keeps it in their head becomes a bottleneck.

9 Continuous Improvement as a Discipline

The JD makes this an explicit responsibility: *“Establish best practices and drive continuous improvement for improving test deployment, test runtime, and yield.”* This is where you go from “runs the tests” to “makes the test process better” — the senior mandate.

Pick projects from data, not vibes. The fleet results tell you where the pain is: the top Pareto failure mode (yield), the slowest test step (runtime), the test that deploys slowly (deployment), the coverage gap that’s letting escapes through. Let the data nominate the project; that also makes the win measurable.

Run it as a closed loop. Baseline the metric → form a hypothesis → make the change → correlate on golden units → deploy → measure the before/after → report. A CI project without a measured delta is just a change. The measurement is what makes it a contribution and what makes the next one easier to get approved.

Examples sized for your first year:

- *Runtime*: replace a fixed, padded soak with a confidence-target BERT (Guide A’s math) — run exactly long enough to prove 1e-12 at 95% and stop. Often a large, safe time win with a statistical guarantee.
- *Yield*: a top failure mode is “PCIe link marginal” with no diagnostic — add lane margining so marginal-but-passing units are caught earlier and real defects come with evidence, *and* set the limit from the fleet distribution instead of a guess.
- *Deployment*: move station-specific settings from code into config so a new board revision or a new CM is a config change, not a code release and re-qualification.

Best practices as artifacts. “Establish best practices” means turning your judgment into things the team reuses: a bring-up checklist, a release/correlation procedure, a failure-triage guide, a standard result schema. Codify it so it survives you and scales to the next hire.

10 What the JD Implies But Doesn't Say

The job description is a paragraph; the *job* is much larger. A senior compute test engineer is expected to bring a lot the JD only gestures at. This chapter makes the implied requirements explicit — each notes the JD phrase it hides behind — so none of them surprises you. Treat it as a checklist of “things I’m quietly expected to be good at.”

10.1 Lab instrumentation & physical-layer measurement (*hidden in “control test instruments”*)

- Bench-instrument fluency and *what each proves on a compute board*: rail voltages under load, ripple/noise, power-up sequencing, inrush, PERST#/reset timing (Guide A’s Instruments chapter). Not “I’ve seen a scope” — “I scope the rail AC-coupled under the load profile that triggers the AER burst.”
- Instrument automation stack: VISA/SCPI over LAN/LXI/USB/GPIB, and a shared-instrument server for multi-station benches (your `equipment_rpc.py`).
- Measurement discipline: 4-wire/Kelvin for low-R and shunts, settling/averaging, fixed vs autorange, and calibration/NIST traceability — log instrument IDN + cal status with every measurement, because a number from an out-of-cal instrument is not a result.
- The instinct that “digital failures are often power/SI failures” — reaching for the scope + AER decode together to root-cause an intermittent link error.

10.2 Software engineering at scale (*hidden in “develop scripts and code”, “build and release”*)

- Not “can write Python,” but test-framework engineering: a config-driven harness, pytest fixtures/parametrize, structured result schemas, defensive coding with timeouts on every instrument/DUT call, graceful failure/recovery.
- C/C++ for tight loops where Python can’t keep up (the AER clear-and-count, the BERT inner loop) — the JD’s “C++ or C# beneficial.”
- Config-over-code so a new board revision or a new CM is a config change, not a code release + re-qualification.
- Version control, release engineering, rollback for *test programs as released artifacts*.

10.3 Data systems, statistics & continuous improvement (*hidden in “analyze results”, “yield”*)

- SPC, C_{pk}/P_{pk} , GR&R/MSA, FPY/RTY, guard-banding, data-driven limit setting — the statistical backbone of “improve yield” (Guide A’s Mass-Production chapter has the detail). This is the difference between “I set the limit at 50 mV” and “the fleet is 20 ± 5 mV, C_{pk} against 50 mV is 2.0, here’s the histogram.”
- A results database + dashboards at fleet scale; Pareto + RCA (5-whys/fishbone) to pick and close the top failure modes; closed-loop CI (baseline → change → correlate → deploy → measure delta).
- MES / test-data-management integration awareness (check-in/out, push verdicts + parameters).

10.4 Working across all four phases & influencing the design (*hidden in “all phases”, “validate new products”*)

- Understanding the coverage envelope of PCBA methods (AOI/AXI/ICT/JTAG/flying probe) even though the CM runs them, so you can allocate coverage across phases (the 10× placement skill).
- DFT (Design-for-Test) influence on the design — pushing *upstream*, while EE is still laying out the board, for test pads/ICT access, boundary-scan/JTAG coverage, a scratch/ID register on custom FPGA cards, accessible rails, telemetry hooks.
- Bring-up of custom hardware with no vendor test plan — working from the schematic (root-port mapping, bifurcation, retimers, rails) to stand up coverage from scratch.

10.5 Reliability/stress integration (*implied by “all phases” + safety-critical compute*)

- Knowing where HALT/HASS/ESS/burn-in/thermal-cycling belong (HALT = design tool; HASS/ESS/burn-in = production screens) and that the test engineer’s job is the in-soak functional monitor — the at-temperature link/error/throttle checks during the screen.
- The “passes at 25 °C, fails at 85 °C” intuition (SerDes margin loss + Arrhenius) and therefore testing hot.

10.6 Safety-criticality mindset & traceability (*inherent to a robotaxi role; not in the JD*)

- “Ship only good units” as non-negotiable: never recover yield by raising the escape rate on safety-critical compute.
- ISO 26262 awareness: ASIL levels, mandatory traceability, coverage/limits as safety-case evidence, change control on test programs/limits.
- Full traceability/genealogy expectation: serial → genealogy → program version → parameters → disposition as an auditable chain.

10.7 Behavioral / seniority expectations (*hidden in “work independently, manage priorities”*)

- Judgment and ownership over an ambiguous mandate (“this board needs coverage”) rather than ticket-taking; the coverage/runtime/yield triad traded *consciously, with data*.
- Evidence-based communication (decoded, layer-identified failures EE believes; altitude-matched updates), influence without authority, disagree-and-commit, and writing it down so knowledge scales across sites.
- Operator/CM-centric design: operators and CM engineers are your *users*; an operator-hostile test fails in practice no matter how technically correct.
- Not breaking the line: read-only-first, sandbox-before-production, golden-unit gate, single-variable debugging, rollback.

How to use this list. These are the unspoken bar. You don’t recite them — you *demonstrate* them: a failure report that’s already decoded and layer-identified; a limit backed by a distribution and a C_{pk} ; a test that’s robust enough to run unattended at a CM; a bring-up done from a schematic with no vendor plan. Every one of them is “senior” rendered as a concrete behavior.

11 Landing the Tools You Bring

You're walking in with a real toolkit (the PCIe BERT, AER decode, lane margining, the NVMe/GPU/GMSL checks, the harness). Done right, it's a fast credibility win; done wrong, it's "the new person who ignored our stack." The sequence that works:

1. Learn their framework first. Before proposing your tool, understand what Zoxx already has. They may already have a BERT, a harness, a dashboard. Adopt their conventions; your goal is to *add to* their stack, not replace it.
2. Find the genuine gap. Your strongest adds are likely the confidence-target BERT (runtime + rigor), lane-margining-based limits (a coverage upgrade over pass-on-link-up), and the clean AER-decode-with-evidence habit. Lead with the gap their current process has, not with "I built a thing."
3. Demo in mock mode, framed as a question. "I built this at home to keep my PCIe skills sharp — it runs the BERT to a confidence target and does receiver lane margining. Could something like this help with X?" is collaborative. "Replace your tool with mine" is not. The mock backend means you can show it working on your laptop on day one with zero risk.
4. Get a senior engineer's buy-in early. Make an ally of whoever owns the current framework. Contribute your tool *into* their structure, with their review, on their release process. Shared ownership beats a parallel tool nobody else maintains.
5. Be gracious about "we already do that." Sometimes the value is one piece (the confidence math, the margining) inside their existing flow, not the whole tool. Take the win that's real. Contributing a useful piece to the team's tool earns more than insisting on your whole tool.

The toolkit's real day-one value isn't "deploy it to the line tomorrow" — it's proof of how you think (read-only-safe, mock-testable, evidence-attached, data-driven limits) and a concrete artifact for the "what would you build" conversations. That's worth a lot even if not one line ever runs on a Zoxx station.

12 Pitfalls to Avoid

The common ways a strong senior hire stumbles in the first months — forewarned:

- Rewriting everything immediately. You don't have the context yet. Earn it first; the existing system has reasons. Ship small, then reshape.
- Breaking the line. A careless test release that false-fails good units is the fastest trust-destroyer there is. Golden-unit gate + release process + rollback, every time.
- "It failed" with no evidence. Forces others to reproduce and erodes your credibility. Always attach decoded, layer-identified evidence.
- Over-promising on schedule. "No surprises" beats "optimistic then late." Flag risk early with options. Senior = predictable.
- Shotgun debugging. Changing many things under pressure. Single variable, named root cause, verified fix — or it comes back.
- Chasing yield by loosening limits without checking escapes. Recovering yield by letting defects through is the cardinal sin in a safety-critical product.
- Treating operators/CMs as afterthoughts. They're your users. A test that's technically correct but operator-hostile fails in practice.
- Version confusion. "Test broke" after a firmware/image update — pin and log versions so you can instantly tell test-regression from HW/FW change.
- Hero culture. Keeping knowledge in your head makes you a bottleneck and a single point of failure. Write it down; make yourself replaceable, which paradoxically makes you promotable.
- Tool-first instead of problem-first. Bringing a solution looking for a problem. Find the real gap, then apply the tool.

13 Living the Leadership Principles (For a Test Engineer)

Zoox operates within Amazon, so the Leadership Principles are the cultural backbone and showed up in your interviews. The point isn't to recite them — it's to *live* them in concrete test-engineering behavior:

- Customer Obsession — your customers are the rider's safety and the operator/CM who runs your test. "Ship only good units" is customer obsession for a robotaxi; an operator-proof, fast, clear test is customer obsession for the line.
- Ownership — own the test where you don't own the line (CMs), own the failure to root cause, own the outcome not just the task. Think beyond "my script ran."
- Dive Deep — root cause, not symptom; data not anecdote; read the registers and the distributions. This is your natural strength — make it visible.
- Insist on the Highest Standards — a test that can't catch the defect isn't done; a limit set by guess isn't done; "fixed, cause unknown" isn't done.
- Earn Trust — evidence-based findings, don't break the line, do what you said. Trust is the currency that lets you drive change.
- Bias for Action — the dashboard you built unasked is the canonical example: see the problem, build the fix, show it. Balanced against not breaking the line (action $\neq$ recklessness).
- Invent and Simplify — the confidence-target BERT and margining-based limits are invention; config-over-code and a clean result schema are simplification.
- Are Right, A Lot / Have Backbone; Disagree and Commit — argue the limit with data, then commit to the decision; revisit with new data, don't re-litigate.
- Frugality — you built the dashboard with no budget on a spare PC. Build-vs-buy judgment and doing more with less is a strength here, not a constraint.

When a behavioral question comes (and they come woven into technical discussions, per your study guide), these are lived examples, not memorized values — which is exactly what they're listening for.

14 A Learning Plan for the Platform

You'll learn fastest by doing, but point the curiosity deliberately:

Read (in priority order): the existing test framework's docs and code; the compute platform's design docs and schematics (especially the PCIe topology — root ports, switches, retimers, bifurcation); the BSP/driver and firmware-update docs; the CM release/correlation process docs; and the relevant slices of the PCIe base spec (AER, link training, equalization, lane margining) and NVMe spec (admin commands, log pages, self-test) — you don't read these cover to cover, you read the chapters behind the registers you touch.

Learn the systems: the results database and its schema, the monitoring dashboard, source control and the release/deploy process, the issue tracker, and the lab — bench inventory, instrument calibration system, fixtures, golden units.

Learn from people: shadow a senior test engineer through a real debug; sit with EE during a board bring-up; watch operators run the test (you'll learn more about your test's real UX in an hour of watching than a week of reading); ask the quality team what field returns actually look like.

Build the platform map yourself: on a real unit, pull the full topology and trace a unit through the four phases. The mental model of “what's connected to what and what's tested where” is the foundation everything else hangs on, and building it yourself cements it.

15 What “Good” Looks Like

Concrete bars to aim for, so you know if you’re on track:

At 90 days: You own an interface’s test (likely PCIe). You’ve closed real hardware bugs to named root causes. Your manager hands you ambiguous, important problems without worrying. You ship through the release process safely and haven’t broken a line.

At 6 months: You own a board’s or a generation’s test program end to end. You’ve driven at least one measured continuous-improvement win (runtime, yield, or deployment). You’ve correlated a test across a CM. You’re the person the team asks about PCIe and high-speed link failures.

At 1 year: You shape compute test *strategy*, not just execute it — coverage allocation across phases, margining-based limits, confidence-target soaks. You’ve brought a new capability to the team that’s now standard practice. You mentor newer engineers. There are measurable yield and runtime improvements with your name on them. EE and SW seek you out during bring-up because your evidence makes their job faster.

That arc — useful, then trusted, then shaping the strategy — is the whole job. You’ve walked a version of it before at smaller scale. Walk it again, bigger, and you’ll have done exactly what they hired a senior engineer to do.

Go get it.